

[Exercise 1]

Report your output result by screenshots and explain the following terms (including how they are calculated) in the output result.

- Total Received Flit
- Total Received Packet
- Global Average Delay
- Network throughput
- Average IP throughput

```
Loading configuration from file "../config_examples/default_config.yaml"... Done
Loading power configurations from file "power.yaml"... Done
Reset for 1000 cycles... done!
Now running for 10000 cycles...
Noxim simulation completed. (11000 cycles executed)

% Total received packets: 1516
% Total received flits: 12124
% Received/Ideal flits Ratio: 1.05243
% Average wireless utilization: 0
% Global average delay (cycles): 11.8047
% Max delay (cycles): 65
% Network throughput (flits/cycle): 1.34711
% Average IP throughput (flits/cycle/IP): 0.0841944
% Total energy (J): 2.12678e-06
%   Dynamic energy (J): 1.51528e-07
%   Static energy (J): 1.97525e-06
[ts110062222@linuxcad30 bin]$
```

Total Received Flit:

A "flit" (flow control unit) is the smallest unit of data transferred in the network. This term refers to the total number of flits successfully received at all destinations in the network.

Total Received Packet:

A "packet" consists of multiple flits (e.g., header, payload, and tail flits). This term refers to the total number of complete packets received at all destinations.

Global Average Delay:

The average delay, measured in cycles, for all packets to travel from their source to destination in the network. It includes queuing, routing, and transmission delays.

Network throughput:

The rate at which flits are successfully transferred across the network, measured in flits per cycle.

Average IP throughput:

The average rate of flits received per cycle at each IP (endpoint) in the network. This is a per-IP metric.

[Exercise 2]

Create a `my_config.yaml` (from `default_config.yaml`), change modify some parameters and re-run the simulation by using:

```
$ ./noxim -config ../config_examples/my_config.yaml
```

Now, monitor the effect, and see how the following parameters will affect the output result mentioned in Exercise 1. Please also explain the reason.

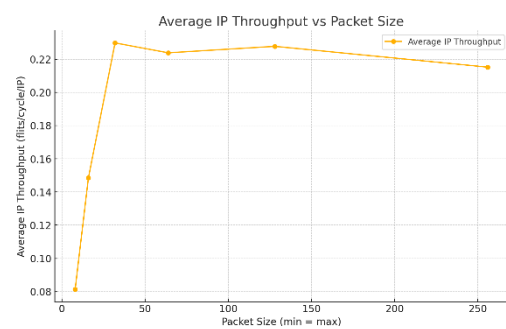
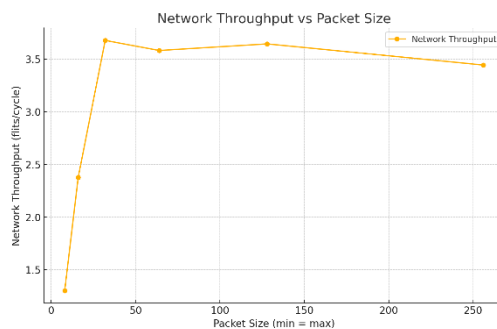
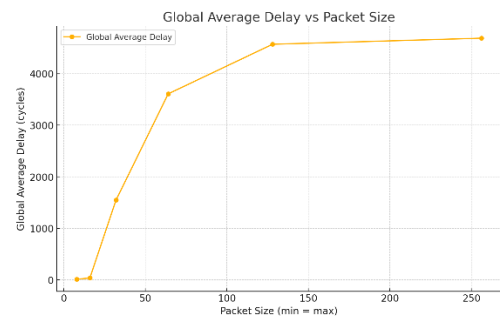
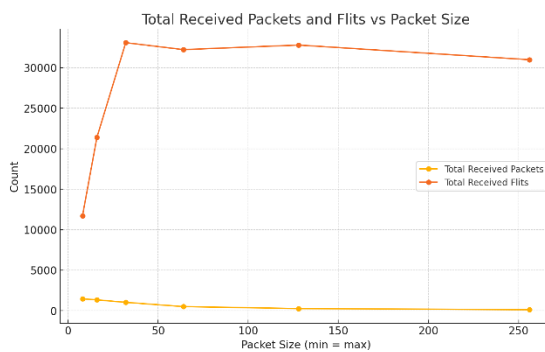
- `packet_size` (min & max)
- `packet_injection_rate`
- `simulation_time`

packet_size

(min, max) = {(8, 8), (16, 16), (32, 32), (64, 64), (128, 128), (256, 256)}

packet_injection_rate: 0.01

simulation_time: 10000



When packet size increases:

- As packet size increases, the total number of packets decreases significantly, but the total number of flits remains relatively stable. This is because larger packet sizes encapsulate more flits per packet.
- Delay increases sharply with larger packet sizes due to greater queuing and contention within the network as larger packets take longer to transmit.
- Throughput initially rises with packet size but begins to stabilize for larger sizes, reflecting a balance between higher flit transmission rates and

increased delays.

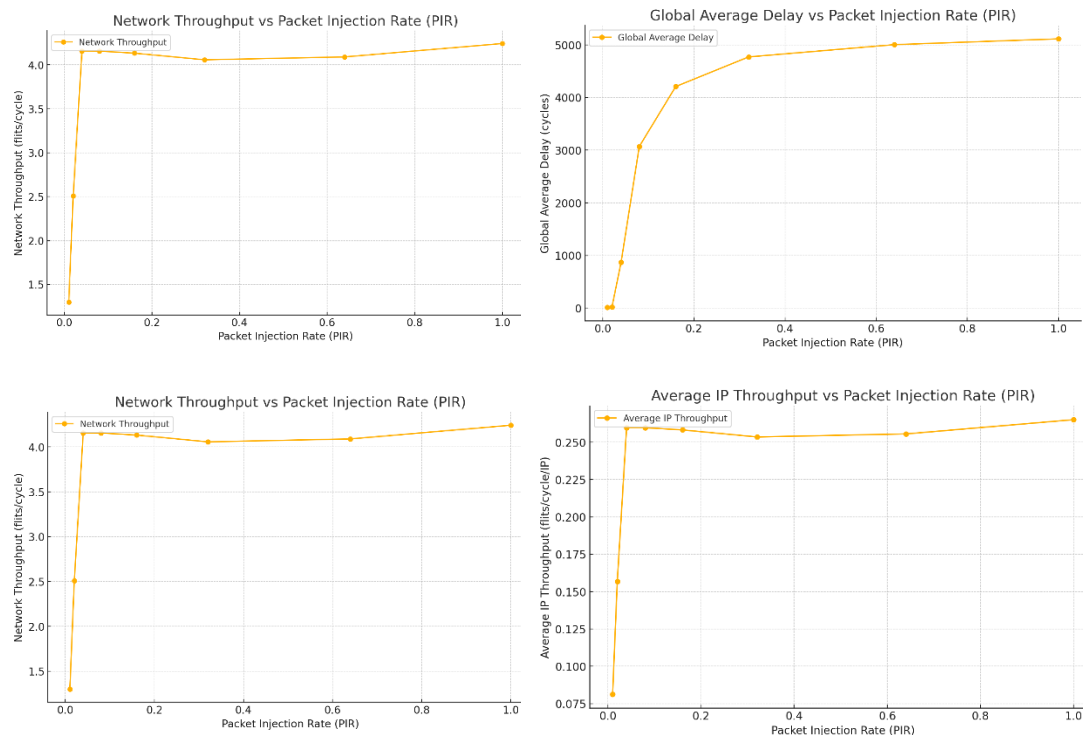
- IP throughput shows a similar pattern, stabilizing after an initial rise, indicating consistent resource utilization per IP despite larger packet sizes.

packet_injection_rate

(min, max) = (8, 8)

packet_injection_rate: {(0.01), (0.02), (0.04), (0.08), (0.16), (0.32), (0.64), (1.0)}

simulation_time: 10000



When packet_injection_rate increases:

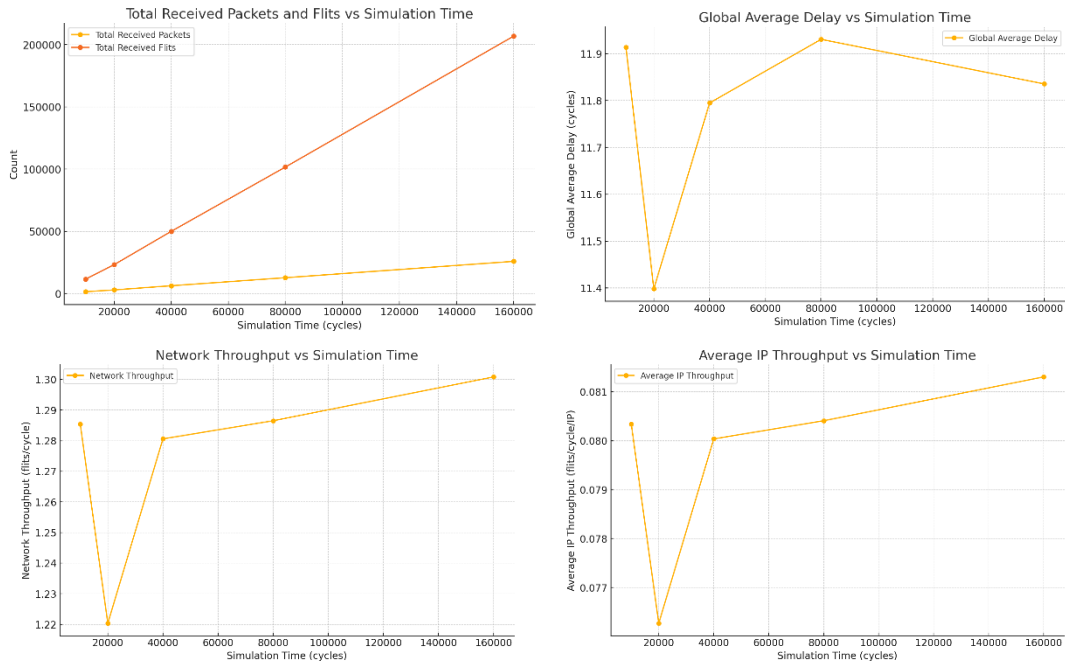
- Total received flits and packets both increase with PIR initially, stabilizing as the network reaches its capacity limit.
- Delay grows gradually at low PIR values but rises steeply beyond the saturation point, indicating network congestion.
- Throughput increases with PIR, stabilizing as the network approaches its peak capacity, with slight variations at higher PIR values.
- IP throughput follows a similar trend, increasing initially and then stabilizing, showing efficient load balancing among IPs.

simulation_time

(min, max) = (8, 8)

packet_injection_rate: 0.01

simulation_time: {(10000), (20000), (40000), (80000), (160000)}

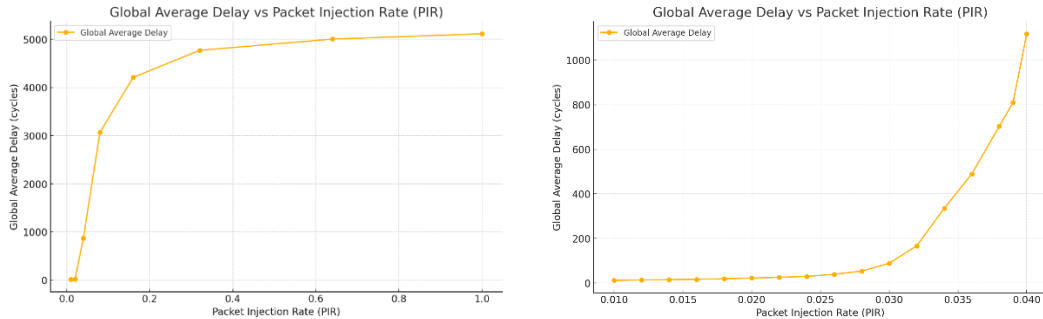


When simulation_time increases:

- As the simulation runs longer, more packets are injected into and transmitted by the network, leading to a proportional increase in the total number of received packets and flits. This linear growth indicates that the network operates consistently without significant congestion or performance degradation over time.
- A stable global average delay suggests that the network is operating efficiently, and increasing the duration of the simulation does not introduce additional delays.
- Network throughput depends on the rate of data transmission per cycle. Since the network maintains steady performance and avoids congestion over longer simulations, throughput remains stable.
- Average IP throughput reflects how well the load is balanced across individual IP nodes. The consistent throughput suggests that all IPs are handling their traffic effectively without overloading or underutilizing specific nodes.

[Exercise 3]

The quality of a NoC can be measured by its saturation point. Plot the curve showing the relationship between PIR and average delay and indicate the saturation point.



1. Graph 1 (PIR Range 0 to 1):

The **Global Average Delay** increases sharply up to a **saturation point** and then stabilizes at high PIR values.

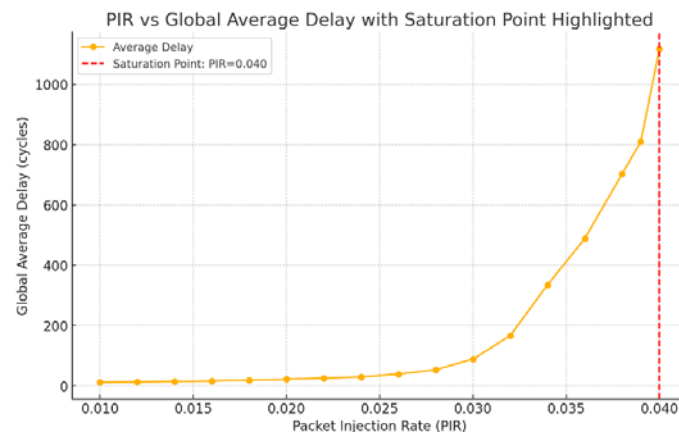
Beyond the saturation point, the delay levels off, indicating that the network cannot process additional traffic efficiently.

2. Graph 2 (Zoomed-In PIR Range 0.01 to 0.04):

The **sharp rise in delay** begins between **PIR = 0.03** and **PIR = 0.04**, indicating the **saturation point** of the NoC.

This is the critical point where the network starts to become congested, resulting in exponentially increasing delays.

The **saturation point** is the PIR value at which the delay begins to rise significantly, signaling that the NoC has reached its capacity. For the provided data, the saturation point is approximately at **PIR = 0.04** or slightly higher.

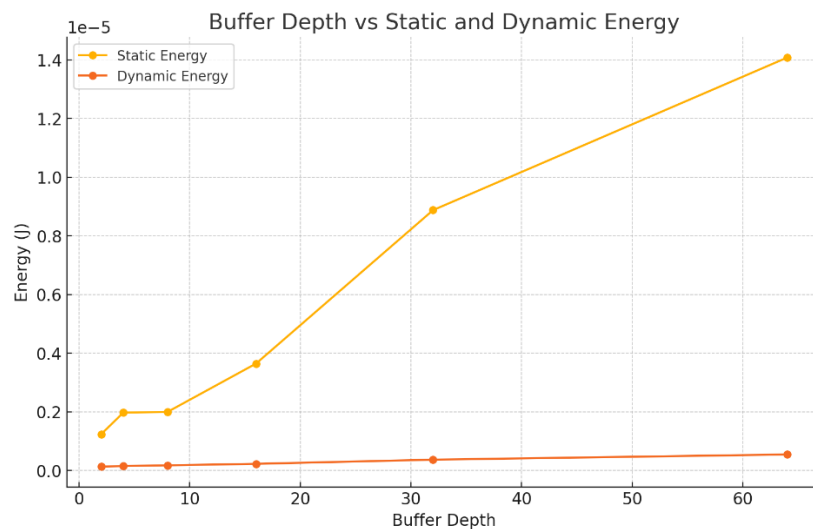
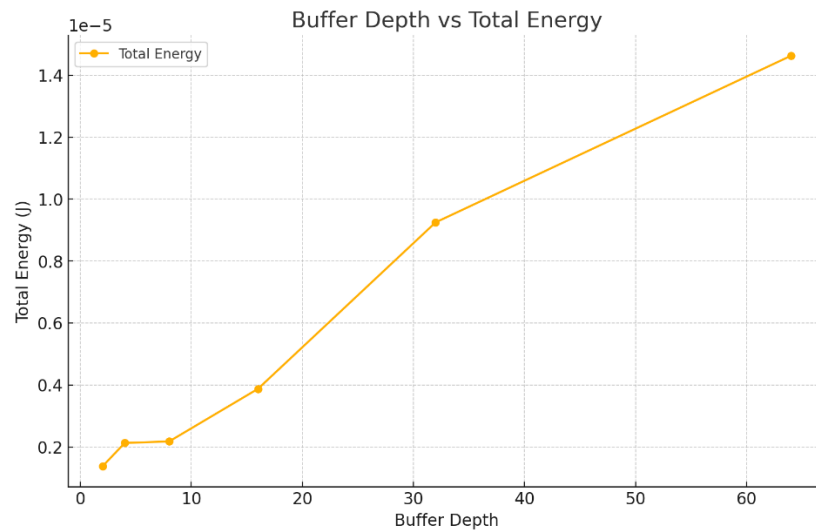


[Exercise 4]

Try to find the optimal power configuration. You can plot figures showing the relationship between your selected parameters and energy to demonstrate the effect. And please also explain the reason of the phenomenon.

I modified the parameter **buffer_depth** and observe the results below:

buffer_depth = {2, 4, 8, 16, 32, 64}



- Total energy consumption increases as the buffer depth increases. The growth is gradual for smaller depths but becomes significantly steeper as the buffer depth reaches higher values.
- Increasing buffer depth provides higher storage but results in higher energy consumption, likely due to the increased static energy costs for maintaining larger buffers.